Demonstrate Cryptographic Fundamentals

Two Primitives Implemented from Scratch in Python: SHA-256 and Diffie-Hellman Key Exchange

Implementations tested against NIST vectors, RFC 3526 and Python's standard library — 12 of 12 tests passed

Project Name	Healthcare IAM Operations Simulation
Organization	Horizon Valley Medical Center (HVMC)
Rubric	Cryptography 1 — Demonstrate Cryptographic Fundamentals (Semester 4, Week 9)
Code	`cryptography_1/crypto_fundamentals/`: `sha256.py`, `diffie_hellman.py`, `test_and_demo.py`, `results.txt`, `results.json`
Environment	Python 3.9.6 on Darwin arm64, 2026-10-13 14:45 — standard library only, no third-party packages
Prepared By	Sabitha S Karanavar
Report Date	October 13, 2026

Rubric Cross-Reference

Rubric requirement	Section	Evidence
Implementation of 2 basic cryptographic primitives using Python	2.1, 3.1, Appendix	SHA-256 (FIPS 180-4) and Diffie-Hellman over the RFC 3526 2048-bit group, written from scratch; 12 unit tests pass
Documentation of security properties for each implementation	2.3, 3.3	Property tables with strength levels, each backed by a measured demonstration
Clear explanation of the mathematical principles used	2.2, 3.2	Bitwise functions, modular addition, Merkle–Damgård; modular exponentiation, cyclic groups, safe primes, discrete logarithm, Miller-Rabin; worked example
Analysis of security guarantees provided by the implementations	2.5, 3.5, 4	What each primitive does and does not guarantee, with attacks demonstrated in code: birthday, preimage, length extension, brute-force discrete log, man-in-the-middle

1. Introduction and Choice of Primitives

A **cryptographic primitive** is a basic, well-studied building block, such as a hash function, a cipher or a key-exchange scheme. Protocols like TLS are built by combining primitives. Two primitives were chosen because they rely on **very different mathematics** and provide **different security goals**, and HVMC needs both:

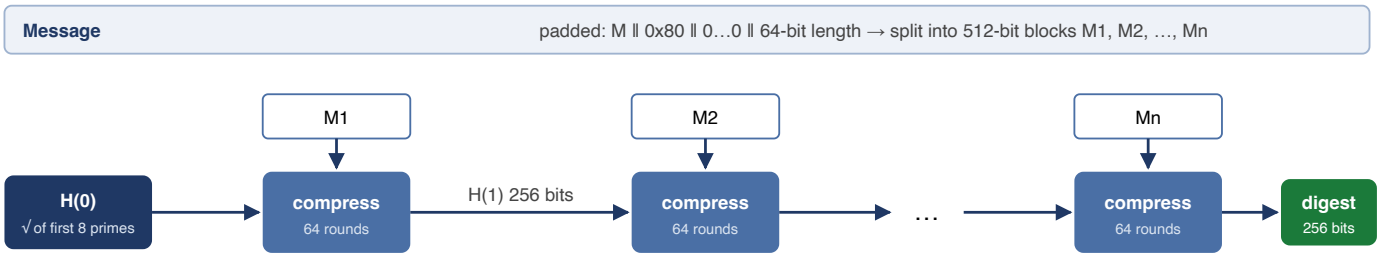
Primitive	Category	Security goal	Mathematical basis	HVMC use case
SHA-256	Keyless hash function	Integrity (detect any change)	Boolean functions, modular addition mod 2^{32}, iterated compression	Detect tampering with lab results, EHR exports and audit-log files; fingerprint the ePHI files in `hvmcehrstorage`
Diffie-Hellman	Public-key key agreement	Confidentiality of a shared key over a public network	Modular exponentiation in a cyclic group; hardness of the discrete logarithm	Let the EHR server and the Laboratory Information System agree on a session key without ever sending it

Both primitives were implemented **from scratch**, using no cryptography library, so that every mathematical step is visible. They also work together: the Diffie-Hellman implementation uses the from-scratch SHA-256 to turn its shared secret into a 256-bit key. This mirrors how TLS 1.3 combines key exchange with a hash-based key derivation. AES, HMAC, password hashing and RSA signatures are left for the Week 10 cryptography rubrics.

2. Primitive 1 — SHA-256 Hash Function

2.1 Implementation

`sha256.py` follows FIPS 180-4 directly. It has four parts: (1) **padding** the message to a multiple of 512 bits, (2) a **message schedule** that expands each 16-word block to 64 words, (3) **64 rounds** of mixing, and (4) a **feed-forward** that adds the result back into the chaining state. The `state` / `prior_len` parameters exist only for the length-extension demonstration in 2.5.



Merkle–Damgård chaining: each block updates the 256-bit chaining value; the final value is the hash.

Figure 1 — How SHA-256 processes a message (Merkle–Damgård construction).

```
def pad(message_len_bytes):
    """Merkle-Damgard padding: 0x80, zeros, then the 64-bit message length in bits,
    so the padded length is a multiple of 64 bytes (512 bits)."""
    bit_len = message_len_bytes * 8
    padding = b"\x80" + b"\x00" * ((55 - message_len_bytes) % 64)
    return padding + bit_len.to_bytes(8, "big")

def compress(state, block):
    """The SHA-256 compression function: mixes one 512-bit block into the 256-bit state."""
    # 1. Message schedule: expand 16 words into 64.
    w = [int.from_bytes(block[i:i + 4], "big") for i in range(0, 64, 4)]
    for t in range(16, 64):
        w.append((small_sigma1(w[t - 2]) + w[t - 7] + small_sigma0(w[t - 15]) + w[t - 16]) & MASK32)

    # 2. 64 rounds over the eight working variables.
    a, b, c, d, e, f, g, h = state
    for t in range(64):
        t1 = (h + big_sigma1(e) + ch(e, f, g) + K[t] + w[t]) & MASK32
        t2 = (big_sigma0(a) + maj(a, b, c)) & MASK32
        h, g, f, e, d, c, b, a = g, f, e, (d + t1) & MASK32, c, b, a, (t1 + t2) & MASK32

    # 3. Davies-Meyer feed-forward: add the input state back (mod 2^32).
    return [(x + y) & MASK32 for x, y in zip(state, [a, b, c, d, e, f, g, h])]

def sha256(message, state=None, prior_len=0):
    """Return the 32-byte SHA-256 digest of `message`.

    `state` and `prior_len` let the hash continue from a known internal state.
    That is only used to demonstrate the length-extension attack.
    """
    if isinstance(message, str):
        message = message.encode()
    h = list(H0 if state is None else state)
    data = message + pad(prior_len + len(message))
    for i in range(0, len(data), 64):
        h = compress(h, data[i:i + 64])
    return b"".join(x.to_bytes(4, "big") for x in h)
```

2.2 Mathematical principles

Words and modular addition. All arithmetic is on 32-bit words. Addition is **modulo 2^{32}** (`& MASK32` discards the carry out of bit 31). Modular addition is *non-linear* over XOR, because the carries mix bits in a way XOR alone cannot, and that is the main source of SHA-256's strength.

Bitwise functions.

Function	Definition	Purpose
Ch(x, y, z)	$(x \wedge y) \oplus (\neg x \wedge z)$	"Choose": x acts as a selector between y and z — a non-linear multiplexer
Maj(x, y, z)	$(x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$	"Majority" vote of three bits — non-linear and balanced
Σ0(x), Σ1(x)	$\mathrm{ROTR}^{2} \oplus \mathrm{ROTR}^{13} \oplus \mathrm{ROTR}^{22}$, $\mathrm{ROTR}^{6} \oplus \mathrm{ROTR}^{11} \oplus \mathrm{ROTR}^{25}$	Spread each bit to three positions per round (diffusion)
σ0(x), σ1(x)	$\mathrm{ROTR}^{7} \oplus \mathrm{ROTR}^{18} \oplus \mathrm{SHR}^{3}$, $\mathrm{ROTR}^{17} \oplus \mathrm{ROTR}^{19} \oplus \mathrm{SHR}^{10}$	Mix message words in the schedule so every input bit affects many rounds

Message schedule. For t = 16…63: $W_t = \sigma 1(W_{t-2}) + W_{t-7} + \sigma 0(W_{t-15}) + W_{t-16} \pmod{2^{32}}$

Round function. Each of the 64 rounds computes $T1 = h + \Sigma 1(e) + Ch(e,f,g) + K_t + W_t, \quad T2 = \Sigma 0(a) + Maj(a,b,c)$ then shifts the eight working variables, with a ← T1 + T2 and e ← d + T1.

"Nothing-up-my-sleeve" constants. The 64 round constants K_t are the first 32 bits of the fractional parts of the **cube roots of the first 64 primes**. The initial state H(0) uses the **square roots of the first 8 primes**. Anyone can recompute them, so the designers could not have hidden a backdoor in specially chosen values.

Padding arithmetic. Append one 1-bit (0x80), then k zero bytes with k = (55 − L) mod 64, then the 64-bit length. Total = L + 1 + k + 8 ≡ 0 (mod 64), which is verified for 200 lengths by `test_padding_is_block_aligned`. Including the length ("MD-strengthening") is what lets a collision-resistant compression function make a collision-resistant hash (Merkle–Damgård theorem).

Feed-forward (Davies–Meyer). H(i) = H(i−1) + E(M_i, H(i−1)). Adding the input back makes the compression function one-way. Even someone who could run the 64 rounds backwards cannot recover H(i−1) from H(i).

2.3 Security properties

Property	Meaning	Generic attack cost	Demonstrated in
Preimage resistance (one-way)	Given a digest h, finding any m with SHA-256(m) = h is infeasible	2^{256}	Demo 4 (truncated to 16 bits)
Second-preimage resistance	Given m, finding a different m′ with the same digest is infeasible	2^{256}	Demo 1 (any edit changes the digest)
Collision resistance	Finding any two different messages with the same digest is infeasible	2^{128} (birthday bound)	Demo 3 (truncated 16/20/24 bits)
Avalanche effect	Changing one input bit changes each output bit with probability ½	—	Demo 2 (1,000 trials)
Deterministic, fixed length	Same input → same 256-bit output, for any input size	—	NIST vectors incl. 1 MB input

2.4 Verification and results

Correctness. The implementation matches all four official FIPS 180-4 test vectors, including the 1,000,000-character message. It also equals Python's `hashlib.sha256` on 300 random messages covering every padding case, and HMAC built on it equals Python's `hmac` module.

Demo 1 — HVMC lab result integrity. Changing a potassium result from 4.1 to 7.1 mmol/L (a dangerous value) changes **124 of 256** digest bits:

Original	`15e92e4781cd8a6c698998b5f6dcde9817c5db1a7bfc12a760e089403aad9ed2`
Tampered	`d7b760b0c3642d5f944992254aae25ab4b6f5821337752a81cf4bd5b92d54471`

Demo 2 — Avalanche. Over 1,000 random one-bit flips, the digest changed by a mean of **127.5 bits** (ideal 128). The standard deviation was 8.11 (ideal 8.0 for a binomial with n = 256, p = ½), with a range of 102–158. This is exactly the behavior of a random function.

Demo 3 — Birthday bound. Digests were truncated so collisions become findable. 20 trials per size:

Truncated size	Mean hashes to first collision	Theory $\sqrt{(\pi/2 \cdot 2^n)}$	Ratio
16-bit	329	321	1.02
20-bit	1,299	1,283	1.01
24-bit	5,116	5,134	1.00

Measured values match theory: collisions appear after about $2^{n/2}$ hashes, not 2^n. For the full 256-bit output that is about 2^{128} hashes, which is infeasible.

Demo 4 — Preimage search. Finding an input for a given 16-bit truncated digest took 22,160 tries on average, against an expected 2^{16} = 65,536. The average of 3 trials varies, but the order of magnitude holds. At 256 bits the cost is 2^{256}.

2.5 Analysis of security guarantees

SHA-256 guarantees	SHA-256 does not guarantee
Any change to the data, even one bit, is detected when the digest is compared against a trusted copy	**Confidentiality.** The hash is public and not encryption. Low-entropy inputs (such as an MRN) can be found by guessing.
128-bit collision security and 256-bit preimage security. No attack on full SHA-256 better than generic is publicly known.	**Authentication on its own.** An attacker who can change the data can also recompute the hash, so the digest must be protected (HMAC or signatures).
Fixed-size fingerprints for any amount of data	**Safe password storage.** It is designed to be fast, so use PBKDF2, bcrypt or Argon2 (Week 10).

Demo 5 — Length-extension attack (a real weakness of Merkle–Damgård). A naive MAC computed as SHA-256(secret ‖ message) can be forged. Because the digest *is* the internal state, an attacker who sees the tag for `user=lisa.tran&role=nurse` can continue hashing from it and produce a valid tag for `user=lisa.tran&role=nurse‖padding‖&role=admin`, without ever knowing the secret.

Server accepts forged tag on naive SHA-256(secret ‖ msg)	**YES — forgery succeeded**
Same attack against HMAC-SHA256 (RFC 2104)	**NO — forgery failed**

Conclusion: when a secret key is involved, use **HMAC**, never a bare hash. SHA-3 and SHA-512/256 are immune by design.

Future threats. Grover's quantum algorithm would cut preimage search to about 2^{128}, which is still secure. Quantum collision search gives no practical advantage, so SHA-256 is considered quantum-safe at its current size.

Implementation caveats. The from-scratch code is correct but **4,110× slower** than `hashlib` (111.0 ms vs 0.027 ms for 64 KiB). Production HVMC systems should use `hashlib` or the platform library. This code exists to show the algorithm.

3. Primitive 2 — Diffie-Hellman Key Exchange

3.1 Implementation

`diffie_hellman.py` contains everything needed, built only from Python integers:

- `mod_pow` — square-and-multiply modular exponentiation (Python's `pow()` is used only in tests, as a reference).
- `is_probable_prime` — Miller-Rabin primality test, 40 rounds.
- `rfc3526_group14_prime` — **computes** the standard 2048-bit prime from its published formula, using π calculated with Machin's formula, rather than pasting a hex constant. The result is then checked against the RFC's published hex.
- `DHParty` — key generation with the `secrets` CSPRNG, peer-key validation, shared secret and SHA-256 key derivation.

```
def mod_pow(base, exponent, modulus):
    """Left-to-right square-and-multiply: computes base^exponent mod modulus
    in O(log exponent) multiplications instead of exponent multiplications."""
    if modulus == 1:
        return 0
    result = 1
    base %= modulus
    for bit in bin(exponent)[2:]:
        result = (result * result) % modulus      # square every bit
        if bit == "1":
            result = (result * base) % modulus     # multiply when the bit is 1
    return result

def rfc3526_group14_prime():
    """RFC 3526 section 3: p = 2^2048 - 2^1984 - 1 + 2^64 * ( floor(2^1918 * pi) + 124476 ).
    The digits of pi are a 'nothing-up-my-sleeve' source: nobody could have chosen
    them to hide a weakness."""
    return 2**2048 - 2**1984 - 1 + 2**64 * (pi_fixed_point(1918) + 124476)

class DHParty:
    """One side of a Diffie-Hellman exchange (e.g. the HVMC EHR server or the LIS)."""

    def __init__(self, name, p=P_2048, g=G):
        self.name, self.p, self.g = name, p, g
        self.q = (p - 1) // 2
        # private key a, chosen uniformly from [2, p-2]
        self._private = secrets.randbelow(p - 3) + 2
        # public key A = g^a mod p
        self.public = mod_pow(g, self._private, p)

    def validate_peer(self, peer_public):
        """Reject values that would make the shared secret predictable:
        1, p-1, and anything outside the prime-order subgroup (small-subgroup attack)."""
        if not 2 <= peer_public <= self.p - 2:
            raise ValueError("peer public key out of range")
        if mod_pow(peer_public, self.q, self.p) != 1:
            raise ValueError("peer public key not in the prime-order subgroup")

    def shared_secret(self, peer_public):
        """s = B^a mod p = (g^b)^a = g^(ab) mod p."""
        self.validate_peer(peer_public)
        return mod_pow(peer_public, self._private, self.p)

    def session_key(self, peer_public):
        """Derive a 256-bit symmetric key from the shared secret with SHA-256."""
        s = self.shared_secret(peer_public)
        s_bytes = s.to_bytes((self.p.bit_length() + 7) // 8, "big")
        return sha256(b"HVMC-DH-v1" + s_bytes)
```

3.2 Mathematical principles

Modular arithmetic and the group. Arithmetic is done modulo a prime p. The non-zero residues {1, …, p−1} form a **cyclic group under multiplication**, $\mathbb{Z}_p^*$, of order p − 1. A **generator** g produces group elements by repeated multiplication: $g^1, g^2, g^3, \ldots \pmod p$.

Why both sides get the same key. Exponents multiply, and multiplication is commutative:

$$s = B^a = (g^b)^a = g^{ab} = (g^a)^b = A^b \pmod p$$

Worked example (Demo 6, verified by `test_toy_example_p23`). Take p = 23 and g = 5. Alice picks a = 6, so A = 5^6 mod 23 = 15625 mod 23 = **8**. Bob picks b = 15, so B = 5^{15} mod 23 = **19**. Alice computes 19^6 mod 23 = **2**, and Bob computes 8^{15} mod 23 = **2**. Both have s = 2. An eavesdropper sees only 23, 5, 8 and 19.

The hard problem — discrete logarithm (DLP). Computing $A = g^a$ mod p is fast. Recovering a from A, the "discrete logarithm", has no known efficient classical algorithm for well-chosen 2048-bit p. Security of DH rests on the **Computational Diffie-Hellman (CDH) assumption**: given g^a and g^b, computing g^{ab} is infeasible.

Square-and-multiply. Computing g^a by multiplying g by itself a times would take about 2^{2048} steps. Instead, the exponent's bits are read left to right: **square** for every bit and also **multiply by g** when the bit is 1. That takes at most 2 × 2048 modular multiplications. The asymmetry between this easy direction (milliseconds) and the hard reverse direction (the DLP) is what DH depends on.

Safe primes and subgroups. RFC 3526 group 14 uses a **safe prime** p = 2q + 1 with q also prime, verified by `test_rfc3526_prime_is_safe_prime`. By Lagrange's theorem, every element's order divides p − 1 = 2q, so the only possible orders are 1, 2, q and 2q. The tests confirm g = 2 has order q ($2^q \equiv 1 \bmod p$), a large prime. This rules out the Pohlig–Hellman attack, which breaks the DLP when the group order has only small factors.

Nothing-up-my-sleeve prime. The prime is $p = 2^{2048} - 2^{1984} - 1 + 2^{64} \cdot (\lfloor 2^{1918} \cdot \pi \rfloor + 124476)$. The digits of π cannot be chosen to plant a weakness. The high and low 64 bits are all 1s, which speeds up modular reduction. The implementation computes π to 1,918 bits and reproduces the published prime exactly.

Miller-Rabin. Write $n - 1 = 2^s \cdot d$ with d odd. By Fermat's little theorem, if n is prime then for any a, either $a^d \equiv 1$ or $a^{2^r d} \equiv -1 \pmod n$ for some $r < s$. A composite n fails this for at least ¾ of the possible bases a. So 40 random rounds give an error probability below $4^{-40} = 2^{-80}$. The test correctly rejects the Carmichael number 561, which fools the simpler Fermat test.

3.3 Security properties

Property	Meaning	Demonstrated in
Key agreement over a public channel	Two parties who share nothing beforehand derive the same secret, and the key itself is never transmitted	Demo 7; `test_shared_key_agreement`
Passive-eavesdropper secrecy	Someone who sees p, g, A, B cannot compute s (CDH assumption): about 112-bit security at 2048 bits	Demo 8 (scaling of brute force)
Forward secrecy (ephemeral use)	New a, b every session, so stealing one session's key or a later key reveals nothing about other sessions	Demo 10
Key validation	Rejects 0, 1, p−1, p and any value outside the order-q subgroup (blocks small-subgroup attacks)	`test_rejects_bad_public_keys`
Uniform key output	Hashing s with SHA-256 gives a uniformly distributed 256-bit key suitable for AES-256	`session_key()`
Not provided: authentication	DH alone does not prove *who* you are talking to	Demo 9 (MITM succeeds)

3.4 Verification and results

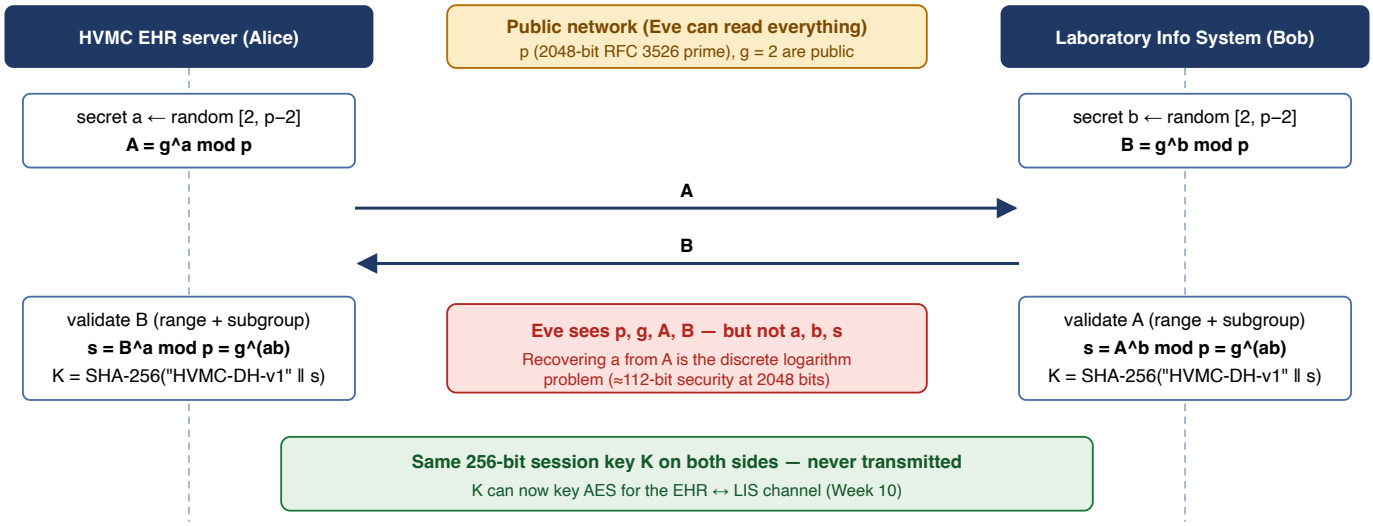


Figure 2 — Diffie-Hellman between the HVMC EHR server and the Laboratory Information System, as implemented.

Correctness. `mod_pow` equals Python's `pow()` on 200 random 512-bit inputs. The computed prime matches RFC 3526 and is a verified safe prime. Five independent exchanges all agreed on the key.

Demo 7 — Full 2048-bit exchange. Both sides derived the identical key `b7496a373a6076029829120fd12fa155ccf5eb20457a5e163cda9ec2f8439d77` (keys match: **True**). Generating two key pairs and two shared secrets took 123 ms in pure Python.

Demo 8 — Brute-forcing the discrete log as the group grows. Random safe primes, 5 private keys per size:

p size	p	Search space q	Mean steps to find key	Mean time	Key correct
16	51,827	25,913	12,965	0.4 ms	Yes
20	1,018,859	509,429	238,801	7.9 ms	Yes
24	16,057,583	8,028,791	3,382,628	113.0 ms	Yes

Every 4 extra bits multiplies the work by about 16, as expected for exhaustive search (about q/2 steps on average). Smarter algorithms exist: baby-step giant-step and Pollard's rho run in $\sqrt{q}$, and the number field sieve is sub-exponential. Even so, NIST SP 800-57 rates a 2048-bit group at **112-bit security**, roughly 2^{112} operations, which is far beyond any attacker today.

3.5 Analysis of security guarantees

Demo 9 — Man-in-the-middle. An attacker ("Mallory") on the network swapped A and B for her own public keys:

EHR and LIS ended up with the same key	**False**
Mallory shares the EHR's key	**True**
Mallory shares the LIS's key	**True**

Mallory can decrypt, read, change and re-encrypt all lab traffic, and neither side notices. **Unauthenticated DH protects only against passive eavesdroppers.** In practice the public values must be signed (RSA/ECDSA signatures with certificates, the Week 10 rubric), as in TLS 1.3, where the server signs the handshake.

Threat / limitation	Analysis	Status in this implementation
Small or weak groups (Logjam, 2015)	512-bit "export" groups were broken in practice, and precomputation against widely shared 1024-bit primes is feasible for nation-states	**2048-bit RFC 3526 group (112-bit security)**
Small-subgroup / invalid-key attacks	A malicious peer sends a value of small order to learn bits of the private key	**Range + subgroup check in `validate_peer`**
Weak randomness	A predictable private key means a predictable session key	**`secrets` module (OS CSPRNG)**
Raw shared secret used as key	s is not uniformly random as a bit string	**Derived with SHA-256 plus a context label**
Man-in-the-middle	No identity binding	**Not addressed — needs signatures/certificates**
Timing side channels	Square-and-multiply does more work for 1-bits, so timing can leak the private key	**Not constant-time (learning code only)**
Quantum computers	Shor's algorithm solves the DLP in polynomial time	**Future: hybrid with ML-KEM (FIPS 203)**

4. Comparative Analysis — How the Two Primitives Work Together

Aspect	SHA-256	Diffie-Hellman (2048-bit)
Type	Symmetric-style, keyless	Public-key (asymmetric)
Goal	Integrity / fingerprinting	Establish a shared secret key
Hardness basis	No known structure to exploit (heuristic; decades of cryptanalysis)	Discrete logarithm problem (number-theoretic assumption)
Classical security level	128-bit (collisions), 256-bit (preimages)	≈112-bit
Quantum outlook	Remains secure (Grover → 128-bit preimage)	Broken by Shor's algorithm → migrate to ML-KEM
Main pitfall shown	Length extension when used as a MAC	Man-in-the-middle without authentication
Speed (pure Python)	111.0 ms per 64 KiB	123 ms per full exchange

Combined use at HVMC. The EHR server and LIS run an ephemeral Diffie-Hellman exchange. SHA-256 turns the shared secret into a 256-bit session key, which would then drive AES-GCM encryption and HMAC-SHA256 integrity (Week 10) for lab traffic. SHA-256 digests of stored lab reports, signed with RSA (Week 10), let clinicians verify a result has not changed since the lab manager approved it. This is the same pattern TLS 1.3 uses: (EC)DHE for keys, HKDF-SHA256 for key derivation, and signatures for authentication.

> **Key lesson:** each primitive is secure only for the job it was designed for. A hash alone does not authenticate, and key exchange alone does not identify the other party. Security comes from combining primitives correctly, using vetted libraries in production rather than custom code.

5. Conclusion

Two fundamental primitives were implemented from scratch in Python 3.9 using only the standard library, and all 12 unit tests pass. SHA-256 reproduces every official test vector and matches `hashlib` bit for bit. Diffie-Hellman derives the RFC 3526 2048-bit prime from π, proves it is a safe prime, and agrees on 256-bit session keys. The mathematics (modular addition and Boolean mixing for SHA-256; cyclic groups, modular exponentiation and the discrete logarithm for DH) was explained and tied to each property. Eleven demonstrations measured those properties, showing the avalanche effect, the birthday bound and exponential discrete-log cost. They also showed the limits of each guarantee, with working length-extension and man-in-the-middle attacks, which lead directly into HMAC and digital signatures in Week 10.

References

- NIST FIPS 180-4, *Secure Hash Standard (SHS)* (2015).
- W. Diffie and M. Hellman, "New Directions in Cryptography," *IEEE Trans. Information Theory* 22(6), 1976.
- RFC 3526, *More Modular Exponential (MODP) Diffie-Hellman groups for IKE* (2003) — group 14.
- RFC 2104, *HMAC: Keyed-Hashing for Message Authentication* (1997).
- NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management* (2020) — security strength of 2048-bit FFC.
- NIST SP 800-56A Rev. 3, *Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography* (2018) — public-key validation.
- D. Adrian et al., "Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice" (Logjam), ACM CCS 2015.
- NIST FIPS 203, *Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM)* (2024).

Appendix A — How to Run

```
cd "cryptography_1/crypto_fundamentals"
python3 test_and_demo.py      # runs 12 unit tests + 11 demos, writes results.txt and results.json
python3 sha256.py             # prints digests of "" and "abc"
python3 diffie_hellman.py     # performs one 2048-bit exchange
```

A.1 Unit test results

Primitive	Test	What it checks	Result
SHA-256	`test_hmac_matches_stdlib`	HMAC built on our SHA-256 equals Python hmac module (50 cases)	**PASS**
SHA-256	`test_matches_hashlib_random`	300 random messages (0–299 bytes) equal hashlib.sha256	**PASS**
SHA-256	`test_nist_vectors`	4 official FIPS 180-4 vectors incl. 1,000,000 × 'a'	**PASS**
SHA-256	`test_padding_is_block_aligned`	Padded length is a multiple of 512 bits for 200 lengths	**PASS**
Diffie-Hellman	`test_generator_order_is_q`	g = 2 has order q: 2^q mod p = 1	**PASS**
Diffie-Hellman	`test_miller_rabin_known_values`	Accepts 2^127−1; rejects Carmichael 561 and a semiprime	**PASS**
Diffie-Hellman	`test_mod_pow_matches_builtin`	Square-and-multiply equals pow() on 200 random 512-bit inputs	**PASS**
Diffie-Hellman	`test_rejects_bad_public_keys`	Rejects 0, 1, p−1, p (small-subgroup / degenerate keys)	**PASS**
Diffie-Hellman	`test_rfc3526_prime_is_safe_prime`	p and q = (p−1)/2 both pass 40-round Miller-Rabin	**PASS**
Diffie-Hellman	`test_rfc3526_prime_known_hex`	Prime derived from π matches published RFC 3526 hex (start and end)	**PASS**
Diffie-Hellman	`test_shared_key_agreement`	5 independent exchanges: both sides derive the same key	**PASS**
Diffie-Hellman	`test_toy_example_p23`	Worked example p=23, g=5 gives A=8, B=19, s=2	**PASS**

A.2 Full program output (results.txt)

```
HVMC Cryptographic Fundamentals — test and demo run
Python 3.9.6 on Darwin arm64, 2026-10-13 14:45

=== Unit tests ===
test_generator_order_is_q (__main__.TestDiffieHellman) ... ok
test_miller_rabin_known_values (__main__.TestDiffieHellman) ... ok
test_mod_pow_matches_builtin (__main__.TestDiffieHellman) ... ok
test_rejects_bad_public_keys (__main__.TestDiffieHellman) ... ok
test_rfc3526_prime_is_safe_prime (__main__.TestDiffieHellman) ... ok
test_rfc3526_prime_known_hex (__main__.TestDiffieHellman) ... ok
test_shared_key_agreement (__main__.TestDiffieHellman) ... ok
test_toy_example_p23 (__main__.TestDiffieHellman) ... ok
test_hmac_matches_stdlib (__main__.TestSHA256) ... ok
test_matches_hashlib_random (__main__.TestSHA256) ... ok
test_nist_vectors (__main__.TestSHA256) ... ok
test_padding_is_block_aligned (__main__.TestSHA256) ... ok

----------------------------------------------------------------------
Ran 12 tests in 4.488s

OK
tests run: 12, failures: 0, errors: 0

=== Demo 1: integrity of an HVMC lab result with SHA-256 ===
original : {"mrn":"HVMC-004417","test":"Potassium","value":"4.1","unit":"mmol/L","verified_by":"Elena Petrova","ts":"2026-10-
13T09:42:00Z"}
digest   : 15e92e4781cd8a6c698998b5f6dcde9817c5db1a7bfc12a760e089403aad9ed2
tampered : value 4.1 -> 7.1 (dangerous hyperkalemia)
```

```
digest   : d7b760b0c3642d5f944992254aae25ab4b6f5821337752a81cf4bd5b92d54471
bits that changed: 124 of 256

=== Demo 2: avalanche effect (1000 one-bit flips) ===
mean bits changed = 127.50 (ideal 128), std dev = 8.11 (ideal 8.0), min = 102, max = 158

=== Demo 3: birthday attack on truncated SHA-256 ===
16-bit hash: collision after 329 hashes on average (theory sqrt(pi/2 * 2^16) = 321)
20-bit hash: collision after 1,299 hashes on average (theory sqrt(pi/2 * 2^20) = 1,283)
24-bit hash: collision after 5,116 hashes on average (theory sqrt(pi/2 * 2^24) = 5,134)
full 256-bit SHA-256: ~2^128 hashes needed -> infeasible

=== Demo 4: preimage search on 16-bit truncated SHA-256 ===
found a preimage after 22,160 tries on average (theory 2^16 = 65,536)
full 256-bit SHA-256: ~2^256 tries needed -> infeasible

=== Demo 5: length-extension attack on naive MAC = SHA-256(secret || message) ===
original message: user=lisa.tran&role=nurse
forged message  :
b'user=lisa.tran&role=nurse\x80\x00\x00\x00
```